שם: יעקב חזון
ת"ז:

תרגיל 4 - תכנות בטוח

ראשית, הופתעתי מאוד לטובה מהתרגיל בית הזה, שכן הוא פותח הזדמנות לחקור, עם חופש יחסי, את עולם החולשות והשמשות שונות.
לאחר מספר שנים שבהם אני עוקב אחר פרסומי חולשות, בעיקר בעולם ההשפעה ולא לפרטי פרטים של המימוש, רציתי לנצל את ההזדמנות על מנת להבין לעומק מנגנון לינוקסי שאיני בקיא בו שיש עליו חולשת RCE ולהבין אותה והשמשה שלה.

לכן, התחלתי בכך שעשיתי חיפוש במאגר ה-CVE עבור "RCE Linux" והתחלתי לעבור על חולשות, את חלקן הכרתי במידה מסוימת או את המנגנון שבו החולשה נמצאה אך חולשה אחת תפסה את העין שלי מאוד בכך שהיא במנגנון שלא עסקתי בו מעולם, לא במהלך עבודתי בצבא ולא בקריאה מעניין אישי - ה-Bluetooth Stack בלינוקס.

ה-"Bluetooth Stack", בדומה למחסנית הרשת המוכרת, הינה הרכיב שמטפל בפרוטוקול ה-Bluetooth (הפיזי - כלומר לאחר קבלת האות הדיגיטלי מהרכיב הפיזי) וגרסאותיו. מכאן, שעליו לדעת לפרסר ולהגיב בהתאם לתקשורת Bluetooth בעבור מנגנון החיפוש, פרסום, יצירת חיבור ותקשורת לאחר חיבור (בין היתר) של פרוטוקול ה-Bluetooth.
החל מגרסאת לינוקס 2.4.6, ששוחררה ב-2005, מחסנית הבלוטות' בלינוקס קרויה BlueZ, שפותחה אצל Qualcomm בהתחלה ולאחר מכן עברה לרישיון GPL.

ניתן לקרוא באתר הרשמי של BlueZ ולהתרשם, אך ניתן לראות שרכיב זה עם הזמן גדל להיות משמעותי מאוד (מעבר לשימוש כמעט מוחלט, הוא גדל מאוד עם הגדילה של Bluetooth, גרסאותיו והפיצ'רים השונים בו). מצד אחד, העובדה שרובו נכתב יחסית בזמן האחרון (יחסית למחסנית הרשת) יכולה להרתיע מלחקור חולשות בו אך עדיין נמצאו בו לא מעט חולשות.

בחרתי לעסוק בסט חולשות Zero-Click בשם BleedingTooth המאפשרות לתוקף בטווח ה-Bluetooth להריץ קוד בהרשאות root על כל רכיבי ה-Bluetooth הלינוקסיים לידו ללא כל התאמתות או מידע נוסף על הנתקף. סט חולשות זה נחקר ופורסם על ידי חוקר אבטחה מ-Google ביולי 2020. החוקר תיעד את תהליך גילוי ומחקר החולשות במאמר הזה והוא מאוד מומלץ לקריאה. הוא היווה חלק מהותי מהמחקר שלי, בנוסף למחקר עצמי על סוגי המתקפות והחולשות שהוא מזכיר והמנגנונים הלינוקסיים המוזכרים.

סט חולשות זה מורכב מ-3 חולשות:
1. CVE-2020-24490 - BadVibes - הצפת זיכרון ב-Heap המאפשרת להריץ קוד ארביטררי כאשר כותבים להפניה לפונקציה שיש בקוד. על מנת לנצלה צריך למצוא את מיקום הפוינטר כדי לדעת כמה לכתוב ואיזה ערך. לשם כך ניתן להשתמש בחולשת זליגת מידע (שתמצא לנו את הערך הדרוש) או סוג חולשה אחר שיאפשר לנו להריץ את הקוד שנכתוב בהצפת זיכרון.
2. CVE-2020-12352 - BadChoice - חולשת זליגת מידע מבוססת Stack. כלומר, חולשה שבאמצעותה התוקף מקבל מידע מה-Stack שהנתקף לא התכוון לשלוח. במקרה זה, זה קורה בגלל שנשלח Struct תגובה שלא כל שדותיו אותחלו ולכן מכילים תוכן קודם של ה-Stack. אם נצליח לכתוב לאזור זה של ה-Stack את הכתובות שמעניינות אותנו ב-Heap, נוכל להשמיש את BadVibes.
3. CVE-2020-12351 - BadKarma - חולשת "בלבול טיפוסים" (כאשר מעבירים טיפוס לא נכון ולאחר מכן מבצעים עליו אופרציות שיכולות לגרום לשגיאות לוגיות ולהתנהגות לא צפויה). גם באמצעותו ניתן להשמיש את BadVibes והוא מספק יריעת תקיפה רחבה יותר.

ניתן לראות את כל קטעי הקוד שאתאר בקוד של Bluez בנתיב root/net/bluetooth
של הריפו של לינוקס החל מגרסא 4.19 ועד גרסא 5.10.

כעת ארחיב על שלושת החולשות.

:BadVibes

חולשה זו נמצאת בקובץ hci_event.c, hci הינו קיצור ל-Host Controller Interface. כינוי זה נמצא בספסיפיקציה של בלוטות' כחלק מממשק אחיד לתקשורת בין הקושחה על רכיבי בלוטות' לבין הדרייבר. ניתן למצוא קישור לחלק המתאים באפיון הממשק פה. ספציפית עבור הקובץ שמעניין אותנו, הוא מממש את הממשק בעבור פאקטות hci (פאקטות בין רכיב ה-BlueZ בקרנל לקושחת בלוטות') בעבור פאקטות מסוג event. כלומר, פאקטות ששולח קושחת הבלוטות' למערכת ההפעלה על אירוע כלשהו. נסתכל על האפיון:

> **HCI Event Packets**: the Host Controller notifies the HCI Driver of events:
>
> - Packet indicator (for UART interfaces) of 4.
> - Event code (8 bits): identifies the event.
> - Parameter length (8-bit): total length of all parameters in bytes.
> - Event parameters: the number of parameters and their length is event specific.

קובץ זה שתיארתי כולל מאות פארסרים לסוגים שונים של אירועים שיכולים להתקבל על ידי רכיב הבלוטות', ניתן לדמיין שעם מגוון גרסאות הבלוטות' והפיצ'רים השונים של כל אחד והעובדה שכל שינוי מצב או מידע שהתקבל מרכיב בלוטות מרוחק שלא התבקש לוקאלית עובר כאירוע, שיהיו הרבה כאלה.

החולשה נמצאה בפארסר של אירוע בשם hci_le_adv_report_evt. אירוע זה אחראי לפרסור אירועים מסוג פרסומת (מטרת האירוע ליידע רכיבי בלוטות' בסביבה על קיום הרכיב ופרטים בסיסיים עליו) בפרוטוקול Bluetooth Low energy. בעבור גרסאות בלוטות' שקדמו ל-5. גודל הפרסומת היה לכל היותר 32 בתים. אמנם, באיפיון של בלוטות' 5 מצוין כי כעת גודל הפרסומת יכול להיות עד ל-255 בתים.

מכאן שעל מנת לא לשנות הרבה קוד, יש ליצור פונקציית פרסור חדשה שתטפל באירוע מסוג זה בעבור עדכון הפרוטוקול. עם זאת, ניתן לראות שבעדכון שמורגג' לתוך לינוקס 4.19, שכחו בפארסר החדש לעשות משהו חשוב, זה הפארסר המקורי:

```
static void hci_le_adv_report_evt(struct hci_dev *hdev, struct sk_buff *skb)
{
        u8 num_reports = skb->data[0];
        void *ptr = &skb->data[1];

        hci_dev_lock(hdev);

        while (num_reports--) {
                struct hci_ev_le_advertising_info *ev = ptr;
                s8 rssi;

                if (ev->length <= HCI_MAX_AD_LENGTH) {
                        rssi = ev->data[ev->length];
                        process_adv_report(hdev, ev->evt_type, &ev->bdaddr,
                                           ev->bdaddr_type, NULL, 0, rssi,
                                           ev->data, ev->length);
                } else {
                        bt_dev_err(hdev, "Dropping invalid advertising data");
                }

                ptr += sizeof(*ev) + ev->length + 1;
        }

        hci_dev_unlock(hdev);
}
```

זה הפארסר הנוסף לאחר העדכון:

```
static void hci_le_ext_adv_report_evt(struct hci_dev *hdev, struct sk_buff *skb)
{
        u8 num_reports = skb->data[0];
        void *ptr = &skb->data[1];

        hci_dev_lock(hdev);

        while (num_reports--) {
                struct hci_ev_le_ext_adv_report *ev = ptr;
                u8 legacy_evt_type;
                u16 evt_type;

                evt_type = __le16_to_cpu(ev->evt_type);
                legacy_evt_type = ext_evt_type_to_legacy(hdev, evt_type);
                if (legacy_evt_type != LE_ADV_INVALID) {
                        process_adv_report(hdev, legacy_evt_type, &ev->bdaddr,
                                           ev->bdaddr_type, NULL, 0, ev->rssi,
                                           ev->data, ev->length);
                }

                ptr += sizeof(*ev) + ev->length;
        }

        hci_dev_unlock(hdev);
}
```

נשים לב להבדל **סופר חשוב** - `if (ev->length <= HCI_MAX_AD_LENGTH)`

בעבור הפארסר המקורי, נבדק שאורך הפרסומת לא גדול מ-32 בתים, אך בחדש, לא נבדק שהוא לא גדול מ-255 בתים ועדיין מעבירים את שניהם לאותה הפונקציה - process_adv_report, שבונה struct עבור הפרסומת.
נסתכל על process_adv_report ונראה שגם שם אין בדיקה על אורך הקלט:

```
static void process_adv_report(struct hci_dev *hdev, u8 type, bdaddr_t *bdaddr,
                               u8 bdaddr_type, bdaddr_t *direct_addr,
                               u8 direct_addr_type, s8 rssi, u8 *data, u8 len)
{
        ...
        if (!has_pending_adv_report(hdev)) {
                ...
                if (type == LE_ADV_IND || type == LE_ADV_SCAN_IND) {
                        store_pending_adv_report(hdev, bdaddr, bdaddr_type,
                                                 rssi, flags, data, len);
                        return;
                }
                ...
        }
        ...
}
```

שם מועבר המידע בפרסומת ואורכו לפונקציה אחרונה בשרשרת שמכניסה את אירוע הפרסומת לתור של פרסומות שלא טופלו (התור נמצא ב-heap):

```
static void store_pending_adv_report(struct hci_dev *hdev, bdaddr_t *bdaddr,
                                     u8 bdaddr_type, s8 rssi, u32 flags,
                                     u8 *data, u8 len)
{
        struct discovery_state *d = &hdev->discovery;
        ...
        memcpy(d->last_adv_data, data, len);
        d->last_adv_data_len = len;
}
```

אך ניתן לראות בהגדרה של hci_dev את התוכן של discovery_state כ-32 בתים וזה לא מתאים למה שיכול לשלוח קושחת הבלוטות' עבור רכיב בגרסאת בלוטות' 5 ומעלה למערכת ההפעלה ולכן לדרוס בתים נוספים בזיכרון! אך האם ב-223 בתים הבאים יש בתים שיהיה לנו מעניין לדרוס? נסתכל כעת על hci_dev:

```
struct hci_dev {
       ...
       struct discovery_state {
              ...
              /* typedef u8 -> __u8 */ unsigned char      last_adv_data[31];          /* 0xab0  0x1f */
              ...
       } discovery; /* 0xa68  0x88 */
       ...
       struct list_head {
              struct list_head * next;                                              /* 0xb18   0x8 */
              struct list_head * prev;                                              /* 0xb20   0x8 */
       } mgmt_pending; /* 0xb18  0x10 */
       ...
       /* size: 4264, cachelines: 67, members: 192 */
       /* sum members: 4216, holes: 17, sum holes: 48 */
       /* paddings: 10, sum paddings: 43 */
       /* forced alignments: 1 */
       /* last cacheline: 40 bytes */
} __attribute__((__aligned__(8)));
```

נבדוק מה זה mgmt_pending. זה שני פוינטרים next ו-prev של רשימה מקושרת דו כיוונית שכל טיפוס בה זה mgmt_pending_cmd שבתוכה פוינטר לפונקציה שנקראת כאשר סיימנו לטפל באירוע hci ואת זה ניתן להטריג למשל על ידי שליחת פאקטת ביטול חיבור (פיצ'ר שנועד לחסוך את המשך הטיפול בחיבור, גם עבור פרסומות, המוטיבציה מובנת אבל זה קצת מריח לא חוסני). נסתכל על זה:

```
struct mgmt_pending_cmd {
        ...
        int                     (*cmd_complete)(struct mgmt_pending_cmd *, u8);

        /* size: 64, cachelines: 1, members: 8 */
        /* sum members: 62, holes: 1, sum holes: 2 */
};
```

כלומר, אנו יכולים לדרוס את הכתובת של הפונקציה שתיקרא לשליטתנו. כעת נשאר לנו להבין מה הכתובת (כמה שיותר מדויק, יכול להיות סטטיסטי) שאליה אנו יכולים להזריק קוד לשליטתנו (למשל בתים שלפני הפוינטר אך במסגרת הזליגת heap הנוכחית). נעשה זאת באמצעות חולשה אחרת.

:BadChoice

חולשה זו נמצאת בקובץ a2mp.c. קובץ זה מתאר את פעולות ה-A2MP (AMP Manager Protocol).
פרוטוקול זה נועד לשליטה על התדר הכי פנוי בטווח התדרים של בלוטות' על מנת שהתקשורת ב-socket הראשי של חיבור הבלוטות' תהיה עם כמה שפחות הפרעות.

בין היתר פרוטוקול זה כולל את האפשרות לבקש מרכיב בלוטות' מרוחק לשלוח לנו את המידע על הבקר AMP שלו וניתן לראות שבמימוש של הפונקציה המטפלת בבקשה הזאת בלינוקס יש בעייתיות:

```
static int a2mp_getinfo_req(struct amp_mgr *mgr, struct sk_buff *skb,
                             struct a2mp_cmd *hdr)
{
        struct a2mp_info_req *req  = (void *) skb->data;
        ...
        hdev = hci_dev_get(req->id);
        if (!hdev || hdev->dev_type != HCI_AMP) {
                struct a2mp_info_rsp rsp;

                rsp.id = req->id;
                rsp.status = A2MP_STATUS_INVALID_CTRL_ID;

                a2mp_send(mgr, A2MP_GETINFO_RSP, hdr->ident, sizeof(rsp),
                          &rsp);

                goto done;
        }
        ...
}
```

הבעייתיות נובעת מזה שאנו מחזירים לתוקף מידע מטיפוס של struct a2mp_info_req אך אנו רק מגדירים ערכים עבור ה-id ועבור ה-status אך עם זאת struct זה מכיל עוד שדות רבים שחוזרים אלינו:

```
struct a2mp_info_rsp {
        /* typedef __u8 */ unsigned char             id;
        /* typedef __u8 */ unsigned char             status;
        /* typedef __le32 -> __u32 */ unsigned int               total_bw;
        /* typedef __le32 -> __u32 */ unsigned int               max_bw;
        /* typedef __le32 -> __u32 */ unsigned int               min_latency;
        /* typedef __le16 -> __u16 */ short unsigned int         pal_cap;
        /* typedef __le16 -> __u16 */ short unsigned int         assoc_size;

        /* size: 18, cachelines: 1, members: 7 */
        /* last cacheline: 18 bytes */
} __attribute__((__packed__));
```

ולכן אם נוכל לכתוב ערכים שמעניינים אותנו ל-stack ככה שכאשר הפונקציה תיקרא לשדות שאינם מאותחלים יהיו את הערכים המעניינים האלה ואז נקבל אותם בחזרה. ערכים מעניינים שכאלה, בהתאם לחולשה הקודמת שדיברנו עליה, יהיו פוינטרים שיעזרו לנו להבין את המבנה זיכרון של הנתקף ואז נוכל לנסות להגיע להרצת קוד עם החולשה הראשונה.

למרות שהחולשה השלישית שאני אציג היא יותר גמישה ומספקת יריעת תקיפה רחבה יותר, גם ההשמשה של החולשה השנייה על מנת ליצור צ'יין עם החולשה הראשונה הייתה לי מרתקת ולכן תיארתי אותה בהמשך.

BadKarma:
חולשה זו גם היא בקובץ a2mp.c. בפונקציה המטפלת בקבלת מידע כלשהו מרכיב הבלוטות' - l2cap_data_rcv:

```
static int l2cap_data_rcv(struct l2cap_chan *chan, struct sk_buff *skb)
{
        ...
        if ((chan->mode == L2CAP_MODE_ERTM ||
             chan->mode == L2CAP_MODE_STREAMING) && sk_filter(chan->data, skb))
                goto drop;
        ...
}
```

הבעיה היא בעיה עיצובית - sk_filter מוגדר לקבל struct sock ומטרת הפונקציה לבדוק תקינות של באפר של סוקט ושאין בו שגיאה. עם זאת, אנו מעבירים לו את ה-struct amp_mgr, בכוונה, כי זה יותר פשוט להסתכל על שניהם בצורה זהה לבדיקה הזאת, אך נוצר פער חוסני.

מקור הפער החוסני הוא ששדות ה-struct sock ממופים לערכים שהוכנסו כטיפוס של struct amp_mgr ולכן אנו יכולים לחפש בשימוש של sk_filter ב-sock ולראות אם אנו יכולים להוביל להרצת קוד בדרך זאת.

מגבלות:
- BadVibes - גרסאת בלוטות' 5 ותמיכה בפרסומות בהגדרות הבלוטות' (ברוב המכישירם מופיע כלהיראות ציבורי).
- BadChoice - קמפול kernel בלי האופציה CONFIG_INIT_STACK_ALL_PATTERN=y, זה לא פופולרי ברוב ההפצות לינוקס בשל האיטיות שבאה עם האופציה הזאת, אך היא גורמת לכך שערכי המחסנית יכתבו על ידי הקרנל בערך כלשהו בכניסה ל-frame.
- BadKarma - אין חוץ מעדכון הקרנל שהזכרתי שסגר את החולשה.

השמשות:
קראתי ולמדתי את ההשמשה של BadKarma ביחד עם BadChoice שמובילים ל-Zero-click RCE מרשים מאוד. ההשמשה מסובכת (לפחות עבורי) ולקח זמן רב להבין אותה.

ראשית, נשים לב שאם אנו כתוקפים מנסים לקבל מידע על מרחב הזיכרון של הנתקף על ידי שימוש ב-BadChoice, אנו נכשלים שכן sk_filter גורם לקרנל פניק.
על כן, הדרך היחידה שלנו להריץ את הפקודה שמובילה לזליגת המידע היא להגדיר את ה-Channel mode כמשהו שהוא לא ERTM או STREAMING (ניתן לראות את זה בקטע קוד האחרון שמצורף ב-BadKarma). על פי האפיון של Bluetooth, זה לא אפשרי (Channel זה כמו פורטים ב-Network Stack).

אמנם, פרקטית במימוש BlueZ, מאפשרים לנו תמיד לשנות את ה-Channel_Mode לסוג השלישי הקיים (יש שלושה) - L2CAP_MODE_BASIC, על ידי סירוב להצעה לקונפיגורציית תקשורת בשלב יצירת החיבור. כלומר, החזרת תגובה בשלב זה של L2CAP_CONF_UNACCEPT וצירוף ה-Channel_Mode המבוקש. ניתן לראות את קטע הקוד בו זה מתבצע:

```
static inline int l2cap_config_rsp(struct l2cap_conn *conn,
                                   struct l2cap_cmd_hdr *cmd, u16 cmd_len,
                                   u8 *data)
{
        struct l2cap_conf_rsp *rsp = (struct l2cap_conf_rsp *)data;
        ...
        scid   = __le16_to_cpu(rsp->scid);
        flags  = __le16_to_cpu(rsp->flags);
        result = __le16_to_cpu(rsp->result);
        ...
        chan = l2cap_get_chan_by_scid(conn, scid);
        if (!chan)
                return 0;

        switch (result) {
        ...
        case L2CAP_CONF_UNACCEPT:
                if (chan->num_conf_rsp <= L2CAP_CONF_MAX_CONF_RSP) {
                        ...
                        result = L2CAP_CONF_SUCCESS;
                        len = l2cap_parse_conf_rsp(chan, rsp->data, len,
                                                   req, sizeof(req), &result);
                        ...
                }
                fallthrough;
        ...
        }
        ...
}
```

שקוראת ל-l2cap_parse_conf_rsp ששם משנה את ה-Channel_Mode למה שרצינו וממשיך בהקמת החיבור (ניתן לראות את זה בשורה האחרונה המצורפת):

```
static int l2cap_parse_conf_rsp(struct l2cap_chan *chan, void *rsp, int len,
                                void *data, size_t size, u16 *result)
{
        ...
        while (len >= L2CAP_CONF_OPT_SIZE) {
                len -= l2cap_get_conf_opt(&rsp, &type, &olen, &val);
                if (len < 0)
                        break;

                switch (type) {
                ...
                case L2CAP_CONF_RFC:
                        if (olen != sizeof(rfc))
                                break;
                        memcpy(&rfc, (void *)val, olen);
                        ...
                        break;
                ...
                }
        }

        if (chan->mode == L2CAP_MODE_BASIC && chan->mode != rfc.mode)
                return -ECONNREFUSED;

        chan->mode = rfc.mode;
        ...
}
```

כעת אנו מכירים דרך לעקוף את הקרנל פאניק ולגרום לזליגת זיכרון, נשאף כעת לגרום לכך ש"בלבול הטיפוסים" בתוך sk_filter יגרום לכך שיורץ קוד בשליטתנו.
נראה את השימושים של ה-struct sock שאליו אנחנו מכניסים טיפוס אחר כי עבור טיפוס זה יש לנו שליטה על מה שאנחנו מכניסים.

```
static inline int sk_filter(struct sock *sk, struct sk_buff *skb)
{
        return sk_filter_trim_cap(sk, skb, 1);
}
// https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/net/core/filter.c
int sk_filter_trim_cap(struct sock *sk, struct sk_buff *skb, unsigned int cap)
{
        int err;
        struct sk_filter *filter;

        /*
         * If the skb was allocated from pfmemalloc reserves, only
         * allow SOCK_MEMALLOC sockets to use it as this socket is
         * helping free memory
         */
        if (skb_pfmemalloc(skb) && !sock_flag(sk, SOCK_MEMALLOC)) {
                NET_INC_STATS(sock_net(sk), LINUX_MIB_PFMEMALLOCDROP);
                return -ENOMEM;
        }
        err = BPF_CGROUP_RUN_PROG_INET_INGRESS(sk, skb);
        if (err)
                return err;

        err = security_sock_rcv_skb(sk, skb);
        if (err)
                return err;

        rcu_read_lock();
        filter = rcu_dereference(sk->sk_filter);
        if (filter) {
                struct sock *save_sk = skb->sk;
                unsigned int pkt_len;

                skb->sk = sk;
                pkt_len = bpf_prog_run_save_cb(filter->prog, skb);
                skb->sk = save_sk;
                err = pkt_len ? pskb_trim(skb, max(cap, pkt_len)) : -EPERM;
        }
        rcu_read_unlock();

        return err;
}
```

לאחר בדיקה מעניינת בתיעוד המחקר על החולשות שבו הגענו למסקנה שאין יותר מדי מה לעשות עם שני השימושים הראשונים ב-sk, נסתכל על השלישי, אליו מועבר sk->sk_filter.
נצרף את המבנה הן של struct sock והן של struct amp_mgr:

```
struct amp_mgr {
        ...
        struct l2cap_conn *        l2cap_conn;                                   /*  0x10   0x8 */
        ...
        /* size: 112, cachelines: 2, members: 11 */
        /* sum members: 110, holes: 1, sum holes: 2 */
        /* last cacheline: 48 bytes */
};
```

```
struct sock {
        struct sock_common {
                ...
                short unsigned int skc_family;                                   /*  0x10   0x2 */
                ...
        } __sk_common; /*     0  0x88 */
        ...
        struct sk_filter *         sk_filter;                                    /* 0x110   0x8 */
        ...
        /* size: 760, cachelines: 12, members: 88 */
        /* sum members: 747, holes: 4, sum holes: 8 */
        /* sum bitfield members: 40 bits (5 bytes) */
        /* paddings: 1, sum paddings: 4 */
        /* forced alignments: 1 */
        /* last cacheline: 56 bytes */
} __attribute__((__aligned__(8)));
```

נשים לב ש-sk_filter נמצא בהיסט של 0x110 מתחילת האובייקט ולכן על מנת להגיע לאן שאנחנו רוצים לשים את הפוינטר שלנו (ככה שהוא יתייחס אליו כ-sk_filter), עלינו לכתוב להיסט 0x10 לאחר האובייקט amp_mgr שאנחנו רושמים אליו, שכן הגודל שלו הוא 112 בתים. בגלל שאנחנו כותבים לזיכרון הקרנל בצאנקים של 128 בתים, אז נצטרך להגיע ככל הנראה לבלוק שלאחר הבלוק בו נשמר האובייקט amp_mgr שלנו.

למזלנו, ניתן למצוא פקודה אחרת בפרוטוקול ה-a2mp שמבצעת כתיבה של תוכן לשליטתנו, בגודל לשליטתנו, לבאפר, ולכן אם נעשה את זה לפני הגדרת amp_mgr, כנראה שנגיע למצב שבו התוכן של הצאנק שלאחריו יהיה התוכן שלשליטתנו (בהסתברות גבוהה וכמעט ודאית בתנאים מבודדים וגם בהסתברות גבוהה במקרה הכללי). אצרף את קטע הקוד בו זה קורה:

```
static int a2mp_getampassoc_rsp(struct amp_mgr *mgr, struct sk_buff *skb,
                                 struct a2mp_cmd *hdr)
{
        ...
        u16 len = le16_to_cpu(hdr->len);
        ...
        assoc_len = len - sizeof(*rsp);
        ...
        ctrl = amp_ctrl_lookup(mgr, rsp->id);
        if (ctrl) {
                u8 *assoc;

                assoc = kmemdup(rsp->amp_assoc, assoc_len, GFP_KERNEL);
                if (!assoc) {
                        amp_ctrl_put(ctrl);
                        return -ENOMEM;
                }

                ctrl->assoc = assoc;
                ctrl->assoc_len = assoc_len;
                ctrl->assoc_rem_len = assoc_len;
                ctrl->assoc_len_so_far = 0;

                amp_ctrl_put(ctrl);
        }
        ...
}
```

כעת, בשאיפה, הגענו למצב בו יש לנו בזיכרון בלוק עם התוכן שהחלטנו עליו בבלוק שאחר האובייקט amp_mgr. ככה אנחנו שולטים בערך שנמצא במקום שאותו sk_filter קורא כ-*struct sk_filter.

ראיתי שלמתקפה זו קוראים controlled out of bounds read. מטורף בעיני לחקור דבר כזה ולהגיע לזה.

כעת אנו יכולים לשלוט בתוכן של ה-heap ולהכין אותה לקראת מתקפת ה-BadKarma וכך לקבל שליטה מלאה על הפוינטר *sk_filter אך על מנת שזה יהיה שימושי, עלינו לגרום לכך שהמצביע יפנה למקום בזיכרון שבו אנו יכולים לשלוט בתוכן שלו. כאן נשתמש בחולשת BadChoice על מנת לפלוט מידע על מבנה הזיכרון וכך להגיע למצב שבו אנו שולטים בבלוק זיכרון (בטכניקה שתיארתי כרגע) במיקום שאנו יודעים (על ידי שימוש ב-BadChoice).

כפי שתיארתי בתיאור החולשה של BadChoice, עלינו לאכלס את האיברים של ה-struct שמוחזר במידע מעניין (ובמקרה שלנו כתובות בזיכרון שהם בשליטתנו). כותב המאמר מציין כי כאשר הוא קרא את הבתים הלא מאוכלסים שהוחזרו, הוא קיבל באופן עקבי כתובת בסגמנט הטקסט השייכת לקרנל הלינוקס. כמו כן, לאחר מספר ניסיונות, הוא גילה שלאחר שליחת פאקטת L2CAP_CONF_RSP ולבקש להעביר את מצב ה-Channel ל-L2CAP_MODE_ERTM לפני כן, אנחנו מקבלים את הכתובת של struct l2cap_chan בהיסט 0x110ב באופן עקבי. לאובייקט זה יש גודל של 792 בתים והוא מאולקץ בצאנק של 1024 בתים.
נסתכל על המבנה של האובייקט:

```
struct l2cap_chan {
        ...
        struct delayed_work {
                struct work_struct {
                        /* typedef atomic_long_t -> atomic64_t */ struct {
                                /* typedef s64 -> __s64 */ long long int counter;        /* 0x110   0x8 */
                        } data; /* 0x110   0x8 */
                        ...
                } work; /* 0x110  0x20 */
                ...
        } chan_timer; /* 0x110  0x58 */
        ...
        /* size: 792, cachelines: 13, members: 87 */
        /* sum members: 774, holes: 9, sum holes: 18 */
        /* paddings: 4, sum paddings: 16 */
        /* last cacheline: 24 bytes */
};
```

כך קיבלנו בדיוק את מה שרצינו וניתן לבצע מתקפת Use-After-Free, כלומר לקבל את הכתובת של אובייקט זה, לאחר מכן, לשחרר אותו על ידי ניתוק החיבור שמגדיר אותו, להתחבר מחדש ולרסס צאנקים של 1024 בתים עם הטכניקה עליה דיברנו כשרצינו לשלוט בתוכן של *sk_filter וכעת להפנות לכתובת זאת על מנת להריץ את הקוד לבחירתנו שכתבנו שם.

חסר לנו חלק אחד - איך אנחנו מריצים קוד לבחירתנו לאחר שליטה על התוכן של *sk_filter, נראה את זה בכך שנצרף חלק מהמבנה שלו:

```
struct sk_filter {
        ...
        struct bpf_prog *          prog;                                 /*  0x18   0x8 */

        /* size: 32, cachelines: 1, members: 3 */
        /* sum members: 28, holes: 1, sum holes: 4 */
        /* forced alignments: 1, forced holes: 1, sum forced holes: 4 */
        /* last cacheline: 32 bytes */
} __attribute__((__aligned__(8)));
```

ובתורו גם struct bpf_prog:

```
struct bpf_prog {
        ...
        unsigned int              (*bpf_func)(const void  *, const struct bpf_insn  *); /*  0x30   0x8 */
        union {
                ...
                struct bpf_insn {
                        /* typedef __u8 */ unsigned char code;                          /*  0x38   0x1 */
                        /* typedef __u8 */ unsigned char dst_reg:4;                     /*  0x39: 0 0x1 */
                        /* typedef __u8 */ unsigned char src_reg:4;                     /*  0x39:0x4 0x1 */
                        /* typedef __s16 */ short int  off;                             /*  0x3a   0x2 */
                        /* typedef __s32 */ int        imm;                             /*  0x3c   0x4 */
                } insnsi[0]; /*  0x38     0 */
        };                                                                              /*  0x38     0 */

        /* size: 56, cachelines: 1, members: 20 */
        /* sum members: 50, holes: 1, sum holes: 4 */
        /* sum bitfield members: 10 bits, bit holes: 1, sum bit holes: 6 bits */
        /* last cacheline: 56 bytes */
};
```

בתוך sk_filter_trim_cap שראינו קוראים ל-bpf_prog_run_save_cb עם filter->prog ונמשיך עם השרשרת לקבל קריאה לכתובת שרשומה שם:

```
static inline u32 __bpf_prog_run_save_cb(const struct bpf_prog *prog,
                                         struct sk_buff *skb)
{
        ...
        res = BPF_PROG_RUN(prog, skb);
        ...
        return res;
}

static inline u32 bpf_prog_run_save_cb(const struct bpf_prog *prog,
                                       struct sk_buff *skb)
{
        u32 res;

        migrate_disable();
        res = __bpf_prog_run_save_cb(prog, skb);
        migrate_enable();
        return res;
}
```

```
#define __BPF_PROG_RUN(prog, ctx, dfunc)        ({                      \
        u32 ret;                                                        \
        cant_migrate();                                                 \
        if (static_branch_unlikely(&bpf_stats_enabled_key)) {           \
                ...
                ret = dfunc(ctx, (prog)->insnsi, (prog)->bpf_func);     \
                ...
        } else {                                                        \
                ret = dfunc(ctx, (prog)->insnsi, (prog)->bpf_func);     \
        }                                                               \
        ret; })

#define BPF_PROG_RUN(prog, ctx)                                         \
        __BPF_PROG_RUN(prog, ctx, bpf_dispatcher_nop_func)
```

```
static __always_inline unsigned int bpf_dispatcher_nop_func(
        const void *ctx,
        const struct bpf_insn *insnsi,
        unsigned int (*bpf_func)(const void *,
                                 const struct bpf_insn *))
{
        return bpf_func(ctx, insnsi);
}
```

ועל כן אנחנו יכולים לשלוט גם ב-RIP כפי שהיינו צריכים על מנת להשלים את כל התמונה.

יש השמשה מטורפת מבחינה טכנית שקראתי אותה והיא גרמה לי להגיד וואו שעושים דברים כאלה.

אני מרגיש חלוק. מצד אחד בחרתי לעסוק במשהו שמאוד עניין אותי ואתגר משמעותי ובאמת למדתי דרכו המון, אך מצד שני אני לא מרגיש שאם הייתי צריך לכתוב השמשה דומה לחולשות האלה אז שיכולתי לעשות את זה בצורה סבירה בגלל פער בניסיון ובידע טכנולוגי מתאים לזה.

אני מקווה שהצלחתי להסביר את זה בצורה טובה ממיטב הבנתי. לא הצלחתי למצוא סטודנט נוסף לעבודה הזאת בשל הרצון שלי לעסוק בסט החולשות הזאת אבל אני מקווה שזה לא מורגש כבעיה באיכות של הדו״ח.

עם זאת, אני מאוד מרוצה שכיסיתי הרבה מאוד נושאים חדשים לי בעולם הנושא של העבודה (המון סוגי מתקפות חדשות ודרכי חשיבה ומחקר חולשות שלא פגשתי בעבר).

בכל מקרה אני מצרף גם את ההשמשה היפה שמצורפת למאמר.

תודה רבה.